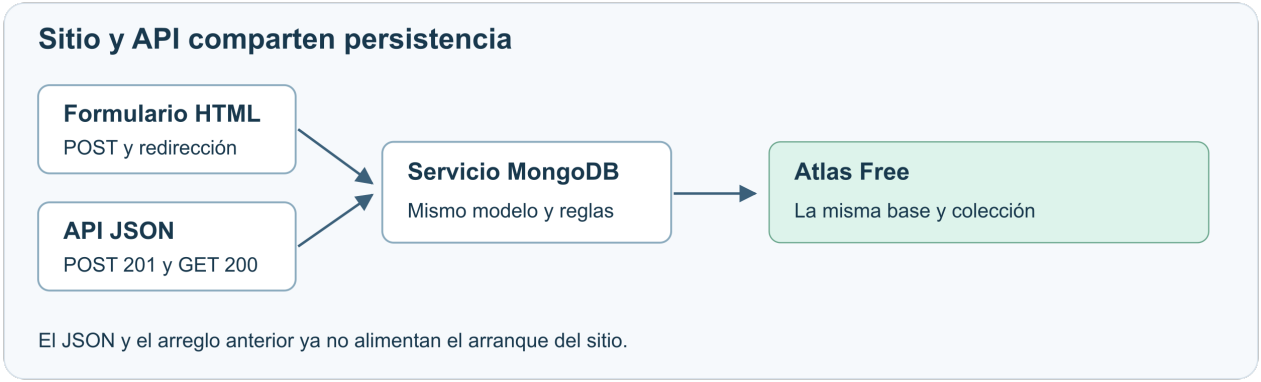
UNIDAD 8

Clase 01 - Creación y lectura desde Express y MongoDB

1. El problema que quedó pendiente

En la unidad 7 el sitio trabajaba con un arreglo y los programas de `scripts/` con MongoDB. Por eso crear un producto en Atlas no lo mostraba necesariamente en la página. En esta unidad cambiamos la fuente de datos del sitio: rutas web y API consultarán el mismo modelo y la misma base.

No comenzamos con un proyecto vacío ni movemos todo a un archivo. Conservamos `src/config/`, `modelos/`, `servicios/`, `controladores/`, `rutas/`, `middleware/` y los programas de terminal en `scripts/`. Tampoco volvemos a incorporar el error del cuerpo ausente: `req.body ?? {}` sigue siendo necesario.



El sitio y los scripts pasan a consultar una misma base.

Los scripts siguen siendo útiles para diagnosticar la conexión. Dejan de ser el único camino para guardar. No se importa un script desde el servidor: esos archivos ejecutan una tarea y cierran recursos.

2. Qué significa crear y leer un recurso

Un recurso es una entidad que identificamos mediante una dirección, por ejemplo un producto. La representación de la API será JSON; la del sitio será HTML. Ambas usan el mismo servicio, pero su contrato HTTP difiere.

Operación	API JSON	Resultado
Listar	GET /api/productos	200 y arreglo; vacío también es válido
Consultar uno	GET /api/productos/:id	200 y producto
Crear	POST /api/productos	201, producto guardado y Location
Filtrar por categoría exacta	GET /api/productos?categoria=Cerámica	200 y coincidencias
Formulario web	POST /productos	302 hacia el listado después de guardar

GET no debe insertar, reiniciar ni cargar una semilla. POST confirma solo después de que termina la escritura. La redirección del formulario evita reenviarlo al actualizar el listado, pero no garantiza que dos envíos independientes no creen duplicados.

Un identificador con formato inválido produce 400. Un ObjectId bien formado que no existe produce 404. La base inaccesible no es «lista vacía»: es un fallo del servicio. Una ruta que no existe también produce 404, pero su mensaje no es el de un documento ausente.

Esta es una API de aprendizaje, no una API pública lista para producción: autenticación, autorización, protección del formulario y despliegue pertenecen a unidades posteriores. No publicar este servidor en Internet como si ya tuviera esos controles.



Un ID inválido produce 400; un ID válido sin documento, 404; un recurso encontrado, 200.

3. Por qué cambiar solo el servicio no alcanza

El servicio anterior devolvía arreglos y objetos inmediatamente. Mongoose realiza operaciones asíncronas. `await` permite obtener el resultado antes de renderizar o responder; no vuelve instantánea la base ni bloquea todo Node.js.



La respuesta espera el resultado y los errores siguen otro camino.

Si se pasa una promesa a EJS, la vista no recibe el arreglo esperado. Si se olvida esperar `create` , puede anunciarse éxito antes de una escritura rechazada. Un controlador declarado `async` devuelve una promesa: con **Express 5**, un rechazo que forma parte de esa promesa llega al manejador de errores. Por eso también debemos esperar o retornar las operaciones internas; una promesa abandonada no queda cubierta.

Los cuatro parámetros `(error, req, res, next)` identifican el middleware de errores. Se coloca después de las rutas y la respuesta de ruta inexistente. No se usa para disfrazar todo error como 200.

Referencia: manejo de errores en Express 5.

4. Paso 1 - Copiar y establecer la línea de base

Qué hacemos y por qué

1. Copiar `mercado-artesanal-mongodb` como `mercado-artesanal-http-mongodb` , sin `.git` , `node_modules` ni secretos. Conservar la entrega anterior.
2. Ejecutar `npm install` con el manifiesto y lockfile de la copia. No actualizar todas las dependencias para hacer esta migración. Verificar `npm ls express mongoose` : este ejemplo usa Express 5. Conservar también las versiones del ZIP de clase si se continúa desde él.
3. Si la copia solo llega al programa de conexión, completar primero el modelo y los scripts de productos de la unidad 7. Si faltan herramientas de calidad, recuperar la instalación y configuración explicadas allí.
4. Revisar el sitio en memoria con `npm start` , sus formularios y 404. Detener el servidor.
5. Reutilizar el clúster **Atlas Free (M0)** y crear un entorno de ejercicio separado mediante el nombre `mercado_artesanal_u8` en la URI. No borrar ni migrar silenciosamente la base de la unidad 7. Crear la base no exige insertar desde el panel: aparecerá al guardar.
6. Mantener la corrección de DNS en `src/config/base-datos.js` : `node:dns` y `dns.setServers(["8.8.8.8", "1.1.1.1"])` antes de conectar, si esos resolutores están autorizados en la red. Conservar el diagnóstico completo y sus límites de la unidad 7. No resuelve contraseñas o IPs no autorizadas.

Estado final de los archivos de este paso

`.env.example`

```
PORT=3000
NODE_ENV=development
MONGODB_URI=mongodb+srv://USUARIO:CLAVE@HOST/mercado_artesanal_u8?retryWrites=true&w=majority
```

`.env`

Archivo local privado. Su estructura es la siguiente; reemplazar los marcadores solo aquí, con host, credenciales codificadas y opciones reales de Atlas:

```
PORT=3000
NODE_ENV=development
MONGODB_URI=mongodb+srv://USUARIO:CLAVE@HOST/mercado_artesanal_u8?retryWrites=true&w=majority
```

No copiar una URI real al material público ni compartir el archivo. `.gitignore` sigue excluyendo `.env` y `node_modules` . La plantilla y el lockfile se conservan en Git. No modificamos `package.json` : usamos el comando directo con `--env-file` al iniciar la versión persistente.

Comprobación

```
node --env-file=.env scripts/comprobar-conexion.js
npm run check
```

La conexión termina sin crear productos. Si falla, resolver entorno, DNS, usuario, IP o red antes de avanzar. Todavía no se cambió el servicio del sitio.

5. Paso 2 - Preparar el modelo y un servicio asíncrono

Qué cambiamos y por qué

Conservamos nombre, categoría, precio y descripción. Agregamos `disponible` con valor predeterminado `true` para observar un default del esquema: no hace falta enviarlo al crear. Se aplica cuando ese campo está `undefined` ; no sustituye arbitrariamente un `null` . No se pide una pantalla para modificarlo.

Mongoose puede convertir un texto numérico como "12000" en Number; un texto no convertible debe rechazarse. El contrato del precio sigue siendo **mayor o igual a 1**, como en el modelo de la unidad 7. Ni la conversión ni un default sustituyen validaciones. Usamos una base nueva: no prometemos que agregar un default reescriba documentos antiguos, especialmente al leerlos con `lean()` .

El servicio nuevo no importa Express ni responde HTTP. Usa el modelo y devuelve una representación simple. MongoDB guarda `_id` ; nuestras vistas ya usaban `producto.id` . En lugar de depender de un virtual de Mongoose que no aparece igual con `lean()` , convertimos `_id` a texto y exponemos `id` explícitamente. El dato en MongoDB continúa siendo un ObjectId, no un entero.

La consulta `find` puede devolver varios documentos; `findById` devuelve uno o `null` . Una Query no es todavía el arreglo: la esperamos. El filtro acepta solo una categoría textual y arma un objeto propio, no utiliza `req.query` como filtro general de MongoDB. La igualdad es exacta, no una búsqueda parcial ni insensible a mayúsculas.

El servicio anterior `src/servicios/productos.js` queda como referencia, **sin editarlo**. En este paso el sitio aún lo utiliza; el servicio MongoDB se conectará en el paso 3. No debemos borrar módulos que todavía importa el arranque.

Estado final de los archivos de este paso

`src/modelos/producto.js`

```
const mongoose = require("mongoose");

const productoSchema = new mongoose.Schema(
  {
    nombre: { type: String, required: true, trim: true },
    categoria: { type: String, required: true, trim: true },
    precio: { type: Number, required: true, min: 1 },
    descripcion: { type: String, required: true, trim: true },
    disponible: { type: Boolean, default: true },
  },
  { collection: "productos" },
);

const Producto = mongoose.model("Producto", productoSchema);

module.exports = { Producto };
```

`src/servicios/productos-mongo.js`

```
const mongoose = require("mongoose");
const { Producto } = require("../modelos/producto");
function representarProducto(documento) {
  return {
    id: documento._id.toString(),
    nombre: documento.nombre,
    categoria: documento.categoria,
    precio: documento.precio,
    descripcion: documento.descripcion,
    disponible: documento.disponible,
  };
}
function crearServicioProductosMongo() {
  async function listar(categoria) {
    const filtro = {};
    if (categoria !== undefined) {
      if (typeof categoria !== "string" || !categoria.trim()) {
        const error = new Error("Categoría inválida");
        error.codigo = "FILTRO_INVALIDO";
        throw error;
      }
      filtro.categoria = categoria.trim();
    }
    const documentos = await Producto.find(filtro).lean();
    return documentos.map(representarProducto);
  }
  async function obtenerPorId(id) {
    if (!mongoose.isObjectIdOrHexString(id)) {
      const error = new Error("Identificador inválido");
      error.codigo = "ID_INVALIDO";
      throw error;
    }
    const documento = await Producto.findById(id).lean();
    return documento ? representarProducto(documento) : null;
  }
  async function crear(datos) {
    const documento = await Producto.create({
      nombre: datos.nombre,
      categoria: datos.categoria,
      precio: datos.precio,
      descripcion: datos.descripcion,
    });
    return representarProducto(documento);
  }
  return { listar, obtenerPorId, crear };
}
module.exports = { crearServicioProductosMongo };
```

Comprobación

```
node --check src/modelos/producto.js
node --check src/servicios/productos-mongo.js
node scripts/validar-producto.js
npm start
```

El modelo válido puede validarse sin conectar. El sitio sigue en memoria: preparar archivos no los conecta automáticamente. No ejecutar todavía el servicio nuevo desde el controlador antiguo. Detener el servidor antes del próximo paso.

Referencias: queries, defaults y ObjectId estricto).

6. Paso 3 - Conectar el sitio con una única conexión

Qué se reemplaza y qué se conserva

Este paso es un cambio coordinado. Preparar todos los archivos siguientes y **reiniciar cuando estén completos**: no mezclar un controlador nuevo con un arranque viejo a mitad del cambio.

En `src/index.js` quitamos `path` , `leerJson` , la ruta al JSON, la carga de productos iniciales y la fábrica del servicio en memoria. Los reemplazamos por conexión y fábrica MongoDB. No borramos el JSON ni `src/archivos.js` : quedan como referencia, pero ya no participan del arranque.

La conexión debe completarse **antes de escuchar el puerto**. Si falla, la aplicación termina sin anunciar disponibilidad. A diferencia de los scripts, no hay un `finally` que desconecte inmediatamente después de `listen` : el servidor sigue atendiendo solicitudes y necesita el pool de conexiones. Se conecta una vez, no una vez por petición. Al recibir SIGINT o SIGTERM, se deja de aceptar conexiones, se esperan las solicitudes activas y se desconecta. Una terminación forzada por el sistema no garantiza ese cierre ordenado; no usamos aquí supervisión avanzada.

En el controlador, listado, detalle y creación esperan el servicio. El detalle deja de aplicar `Number(req.params.id)` . El middleware de entrada admite los campos esperados y alinea el precio con el esquema: el validador antiguo dejaba pasar 0.5 aunque el modelo lo rechazaba. Conservamos valores y respuesta 400 en el formulario, y rechazamos arreglos, objetos o booleanos como precio. La lista explícita también evita aceptar un `_id` o `disponible` enviado por el cliente.

Las vistas no cambian: reciben `id` , nombre, categoría, precio y descripción como antes. El router HTML mantiene `/nuevo` antes de `/:id` . Los recursos públicos, el layout, el middleware de solicitudes y la configuración de entorno/DNS quedan intactos.

El middleware de errores distingue datos inválidos y fallos inesperados, no imprime URIs ni devuelve trazas. Una validación del esquema no atrapada por el filtro HTTP devuelve 400; el formulario conserva valores. JSON mal formado también devuelve 400. Esta separación básica es necesaria ahora; su diseño más amplio se profundiza en la unidad 11.

Estado final de todos los archivos de este paso

`src/index.js`

```
const { leerConfiguracion } = require("./config/entorno");
const {
  conectarBaseDatos,
  desconectarBaseDatos,
} = require("./config/base-datos");
const { crearServicioProductosMongo } = require("./servicios/productos-mongo");
const { crearApp } = require("./app");
async function main() {
  const { puerto, formatoRegistro } = leerConfiguracion();
  await conectarBaseDatos();
  const servicioProductos = crearServicioProductosMongo();
  const app = crearApp({ servicioProductos, formatoRegistro });
  const servidor = app.listen(puerto, () => {
    console.log(`Aplicación disponible en http://localhost:${puerto}`);
  });
  let cerrando = false;
  function cerrar() {
    if (cerrando) return;
    cerrando = true;
    servidor.close(async () => {
      try {
        await desconectarBaseDatos();
      } catch (error) {
        console.error("Error al cerrar:", error.name);
        process.exitCode = 1;
      }
    });
  }
  process.once("SIGINT", cerrar);
  process.once("SIGTERM", cerrar);
  servidor.on("error", (error) => {
    console.error("No se pudo escuchar el puerto:", error.code);
    process.exitCode = 1;
    desconectarBaseDatos().catch((fallo) => {
      console.error("Error al cerrar:", fallo.name);
    });
  });
}
main().catch(async (error) => {
  console.error("No se pudo iniciar la aplicación:", error.name);
  console.error("Revisar entorno, DNS, red, acceso de Atlas y puerto.");
  process.exitCode = 1;
  try {
    await desconectarBaseDatos();
  } catch (fallo) {
    console.error("Error al cerrar:", fallo.name);
  }
});
```

src/middleware/productos.js

```
function prepararAreaProductos(req, res, next) {
  res.locals.seccion = "Productos artesanales";
  next();
}
function validarProducto(req, res, next) {
  const valores = req.body ?? {};
  const nombre =
    typeof valores.nombre === "string" ? valores.nombre.trim() : "";
  const categoria =
    typeof valores.categoria === "string" ? valores.categoria.trim() : "";
  const descripcion =
    typeof valores.descripcion === "string" ? valores.descripcion.trim() : "";
  const precioRecibido = valores.precio;
  const tipoPrecioValido =
    typeof precioRecibido === "number" ||
    (typeof precioRecibido === "string" && precioRecibido.trim() !== "");
  const precio = tipoPrecioValido ? Number(precioRecibido) : NaN;
  if (
    !nombre ||
    !categoria ||
    !descripcion ||
    !Number.isFinite(precio) ||
    precio < 1
  ) {
    if (req.baseUrl === "/api/productos") {
      return res.status(400).json({
        error: "Completá los campos con un precio mayor o igual a 1.",
      });
    }
    return res.status(400).render("productos/nuevo", {
      titulo: "Nuevo producto",
      error: "Completá los campos con un precio mayor o igual a 1.",
      valores,
    });
  }
  req.productoValidado = { nombre, categoria, precio, descripcion };
  next();
}
module.exports = { prepararAreaProductos, validarProducto };
```

`src/controladores/productos.js`

```
function crearControladorProductos(servicio) {
  async function listar(req, res) {
    const productos = await servicio.listar();
    res.render("productos/lista", {
      titulo: "Productos artesanales",
      productos,
    });
  }
  function mostrarFormulario(req, res) {
    res.render("productos/nuevo", {
      titulo: "Nuevo producto",
      error: null,
      valores: {},
    });
  }
  async function mostrarDetalle(req, res) {
    const producto = await servicio.obtenerPorId(req.params.id);
    if (!producto) {
      return res.status(404).render("no-encontrado", {
        titulo: "Producto no encontrado",
        mensaje: "No existe un producto con ese identificador.",
      });
    }
    res.render("productos/detalle", { titulo: producto.nombre, producto });
  }
  async function crear(req, res) {
    await servicio.crear(req.productoValidado);
    res.redirect("/productos");
  }
  async function listarApi(req, res) {
    const productos = await servicio.listar(req.query.categoria);
    res.json(productos);
  }
  return {
    listar,
    mostrarFormulario,
    mostrarDetalle,
    crear,
    listarApi,
  };
}
module.exports = { crearControladorProductos };
```

`src/middleware/errores.js`

```
function crearManejadorErrores({ vistaFormulario, tituloFormulario }) {
  return function manejarErrores(error, req, res, next) {
    if (res.headersSent) {
      return next(error);
    }
    const esValidacion = error.name === "ValidationError";
    const esEntrada =
      esValidacion ||
      error.codigo === "ID_INVALIDO" ||
      error.codigo === "FILTRO_INVALIDO" ||
      error.type === "entity.parse.failed";
    const estado = esEntrada ? 400 : 500;
    let mensaje = esEntrada
      ? "Revisá los datos enviados."
      : "No se pudo completar la operación.";
    if (error.codigo === "ID_INVALIDO") {
      mensaje = "El identificador debe tener 24 caracteres hexadecimales.";
    }
    console.error("Solicitud fallida:", res.locals.solicitudId, error.name);
    if (req.path.startsWith("/api/")) {
      return res.status(estado).json({
        error: mensaje,
        campos: esValidacion ? Object.keys(error.errors) : [],
      });
    }
    if (esValidacion && req.method === "POST") {
      return res.status(400).render(vistaFormulario, {
        titulo: tituloFormulario,
        error: mensaje,
        valores: req.body ?? {},
      });
    }
    return res.status(estado).render("no-encontrado", {
      titulo: esEntrada ? "Solicitud inválida" : "Error del servicio",
      mensaje,
    });
  };
}
module.exports = { crearManejadorErrores };
```

`src/app.js`

```
const express = require("express");
const expressLayouts = require("express-ejs-layouts");
const morgan = require("morgan");
const path = require("node:path");
const {
  crearIdentificadorSolicitud,
  medirDuracion,
} = require("./middleware/solicitudes");
const { crearManejadorErrores } = require("./middleware/errores");
const { crearControladorProductos } = require("./controladores/productos");
const { crearRouterProductos } = require("./rutas/productos");
function crearApp({ servicioProductos, formatoRegistro }) {
  const app = express();
  const controlador = crearControladorProductos(servicioProductos);
  app.set("view engine", "ejs");
  app.set("views", path.join(__dirname, "..", "views"));
  app.set("layout", "layouts/main");
  app.use(morgan(formatoRegistro));
  app.use(crearIdentificadorSolicitud());
  app.use(medirDuracion);
  app.use(expressLayouts);
  app.use(express.static(path.join(__dirname, "..", "public")));
  app.use(express.urlencoded({ extended: false }));
  app.use(express.json());
  app.get("/", (req, res) => {
    res.render("inicio", { titulo: "Mercado artesanal" });
  });
  app.get("/api/productos", controlador.listarApi);
  app.use("/productos", crearRouterProductos(controlador));
  app.use((req, res) => {
    if (req.path.startsWith("/api/")) {
      return res.status(404).json({ error: "Ruta no encontrada." });
    }
    res.status(404).render("no-encontrado", {
      titulo: "Página no encontrada",
      mensaje: "La dirección solicitada no existe.",
    });
  });
  app.use(
    crearManejadorErrores({
      vistaFormulario: "productos/nuevo",
      tituloFormulario: "Nuevo producto",
    }),
  );
  return app;
}
module.exports = { crearApp };
```

Comprobación

```
npm run format
npm run check
node --env-file=.env src/index.js
```

Ahora sí necesitamos la URI al arrancar. `npm start` solo alcanza si el entorno ya proporciona MONGODB_URI; si el manifiesto conserva `start:local` , ese script también carga `.env` . Usar un solo servidor para no ocupar dos veces el puerto.

Abrir inicio, `/productos` y `/productos/nuevo` . En una base nueva el listado estará vacío: eso es correcto. Crear un producto desde el formulario, comprobar redirección 302, listado, detalle con su ID textual y `GET /api/productos` . Detener y volver a iniciar con el mismo comando: debe recuperarse el mismo `id` . No esperar que un ID numérico como `/productos/1` siga funcionando.

La API todavía tiene solo listado en este paso. Mantener la conexión durante dos solicitudes consecutivas demuestra que no se cerró después de la primera.

7. Paso 4 - Exponer creación y lectura como API JSON

Por qué otro router

El router HTML describe páginas; el router API describe recursos JSON. Comparten controlador, servicio y modelo, pero no la redirección ni la plantilla de respuesta. `POST /api/productos` devuelve 201 y la dirección en Location; el cliente puede recuperar exactamente ese recurso con un GET.

Agregar `productos-api.js` , completar el controlador con `obtenerApi` y `crearApi` , y reemplazar en `app.js` el GET aislado de la API por el router completo. **No conservar ambas declaraciones de /api/productos**: la primera podría responder antes de llegar al router.

El filtro de entrada usa `req.baseUrl` para distinguir una creación JSON de un formulario. Todos los errores de la API, incluidos los de rutas inexistentes, son JSON. El manejador queda después de las rutas. Ningún camino cierra MongoDB al responder.

Estado final de los archivos de este paso

`src/rutas/productos-api.js`

```
const express = require("express");
const { validarProducto } = require("../middleware/productos");

function crearRouterProductosApi(controlador) {
  const router = express.Router();
  router.get("/", controlador.listarApi);
  router.get("/:id", controlador.obtenerApi);
  router.post("/", validarProducto, controlador.crearApi);
  return router;
}

module.exports = { crearRouterProductosApi };
```

`src/controladores/productos.js`

```
function crearControladorProductos(servicio) {
  async function listar(req, res) {
    const productos = await servicio.listar();
    res.render("productos/lista", {
      titulo: "Productos artesanales",
      productos,
    });
  }
  function mostrarFormulario(req, res) {
    res.render("productos/nuevo", {
      titulo: "Nuevo producto",
      error: null,
      valores: {},
    });
  }
  async function mostrarDetalle(req, res) {
    const producto = await servicio.obtenerPorId(req.params.id);
    if (!producto) {
      return res.status(404).render("no-encontrado", {
        titulo: "Producto no encontrado",
        mensaje: "No existe un producto con ese identificador.",
      });
    }
    res.render("productos/detalle", { titulo: producto.nombre, producto });
  }
  async function crear(req, res) {
    await servicio.crear(req.productoValidado);
    res.redirect("/productos");
  }
  async function listarApi(req, res) {
    const productos = await servicio.listar(req.query.categoria);
    res.json(productos);
  }
  async function obtenerApi(req, res) {
    const producto = await servicio.obtenerPorId(req.params.id);
    if (!producto) {
      return res.status(404).json({ error: "Producto no encontrado." });
    }
    res.json(producto);
  }
  async function crearApi(req, res) {
    const producto = await servicio.crear(req.productoValidado);
    res.location(`/api/productos/${producto.id}`).status(201).json(producto);
  }
  return {
    listar,
    mostrarFormulario,
    mostrarDetalle,
    crear,
    listarApi,
    obtenerApi,
    crearApi,
  };
}
module.exports = { crearControladorProductos };
```

`src/app.js`

```
const express = require("express");
const expressLayouts = require("express-ejs-layouts");
const morgan = require("morgan");
const path = require("node:path");
const {
  crearIdentificadorSolicitud,
  medirDuracion,
} = require("./middleware/solicitudes");
const { crearManejadorErrores } = require("./middleware/errores");
const { crearControladorProductos } = require("./controladores/productos");
const { crearRouterProductos } = require("./rutas/productos");
const { crearRouterProductosApi } = require("./rutas/productos-api");
function crearApp({ servicioProductos, formatoRegistro }) {
  const app = express();
  const controlador = crearControladorProductos(servicioProductos);
  app.set("view engine", "ejs");
  app.set("views", path.join(__dirname, "..", "views"));
  app.set("layout", "layouts/main");
  app.use(morgan(formatoRegistro));
  app.use(crearIdentificadorSolicitud());
  app.use(medirDuracion);
  app.use(expressLayouts);
  app.use(express.static(path.join(__dirname, "..", "public")));
  app.use(express.urlencoded({ extended: false }));
  app.use(express.json());
  app.get("/", (req, res) => {
    res.render("inicio", { titulo: "Mercado artesanal" });
  });
  app.use("/api/productos", crearRouterProductosApi(controlador));
  app.use("/productos", crearRouterProductos(controlador));
  app.use((req, res) => {
    if (req.path.startsWith("/api/")) {
      return res.status(404).json({ error: "Ruta no encontrada." });
    }
    res.status(404).render("no-encontrado", {
      titulo: "Página no encontrada",
      mensaje: "La dirección solicitada no existe.",
    });
  });
  app.use(
    crearManejadorErrores({
      vistaFormulario: "productos/nuevo",
      tituloFormulario: "Nuevo producto",
    }),
  );
  return app;
}
module.exports = { crearApp };
```

Pruebas con Postman

Usar **Postman de escritorio** para enviar solicitudes al servidor local. Mantener Express iniciado con `node --env-file=.env src/index.js` . Postman actúa como cliente de la API; MongoDB sigue siendo consultado por el servicio del proyecto.

1. Consultar el listado antes de crear

1. Abrir una solicitud HTTP en Postman.
2. Seleccionar el método **GET** y escribir `http://localhost:3000/api/productos` en la URL.
3. En **Body**, dejar **none**: esta consulta no necesita cuerpo.
4. Pulsar **Send**.
5. Revisar el estado **200** y el arreglo en el cuerpo de la respuesta. Si la base es nueva, `[]` es válido.
6. Anotar la cantidad inicial para comparar después de crear y de probar rechazos.

2. Crear un producto desde JSON

Preparar otra solicitud con método **POST** y URL `http://localhost:3000/api/productos` . En **Body**, seleccionar **raw** y elegir **JSON** como formato. Pegar este cuerpo:

```
{
  "nombre": "Cuenco del valle",
  "categoria": "Cerámica",
  "precio": 15000,
  "descripcion": "Pieza artesanal de ejemplo."
}
```

En **Headers de la solicitud**, comprobar `Content-Type: application/json` . Postman configura esa cabecera al seleccionar JSON, salvo que se haya sobrescrito manualmente. Es la cabecera que acompaña al cuerpo enviado y permite interpretarlo con `express.json()` .

Pulsar **Send una sola vez**. Revisar:

- **Estado 201:** la escritura terminó y se creó el producto.
- **Cuerpo de la respuesta:** el producto guardado, con `id` textual y `disponible: true` , aunque no se envió ese campo.
- **Headers de la respuesta:** la cabecera `Location` , que contiene la dirección del recurso creado.

Guardar el ID y el valor de `Location` . Repetir un POST válido solicita otra creación: no reenviarlo para comprobar que el producto sigue existiendo.

3. Recuperar el producto mediante Location

Abrir otra solicitud **GET**, con **Body en none**. Si `Location` devuelve una ruta relativa, como `/api/productos/...` , agregar delante `http://localhost:3000` . La URL debe contener el ID real recibido:

```
http://localhost:3000/api/productos/ID_RECIBIDO
```

Reemplazar `ID_RECIBIDO` por el valor concreto de `id` ; no enviar el marcador. Pulsar **Send** y comprobar **200**, el mismo ID y los mismos campos del producto creado.

Las cabeceras de la solicitud describen lo que enviamos; las de la respuesta describen lo que devuelve Express. `Location` se busca en **Headers de la respuesta**, no en los de la solicitud.

4. Diferenciar un ID inválido de un documento inexistente

Preparar estas consultas **GET** por separado, sin cuerpo:

```
http://localhost:3000/api/productos/no-es-un-id
http://localhost:3000/api/productos/000000000000000000000000
```

La primera debe devolver **400** con JSON de error: el formato no es válido. La segunda usa un formato válido y debe devolver **404 solo si no existe ese documento**. El ID de ceros no significa automáticamente "inexistente".

5. Filtrar por categoría exacta

En una solicitud **GET** a `http://localhost:3000/api/productos` , abrir **Params** y añadir una fila activa con **Key** `categoria` y **Value** `Cerámica` . Enviar y revisar **200** con las coincidencias.

Cambiar el valor por una categoría sin coincidencias: debe devolver **200** y `[]` . Para probar el rechazo de categoría repetida, activar dos filas con la misma Key `categoria` y valores distintos. El servicio debe rechazar esa entrada con **400**. Retirar o desactivar las filas al volver al listado sin filtro.

Estos controles consultan los endpoints ya construidos. La API devuelve `id` textual y no exige que el consumidor use `_id` ni `__v` .

8. Paso 5 - Comprobar el contrato y la persistencia

Matriz de pruebas

Caso	Estado esperado	Evidencia
GET listado	200	Arreglo de la base, no del JSON
GET colección vacía, si es nueva	200	Arreglo vacío sin inserción automática
POST JSON válido	201	ID, documento y Location
GET Location después de crear	200	Mismo ID y campos
GET con ID mal formado	400	JSON de error
GET con ID válido inexistente	404	JSON de recurso no encontrado
POST sin cuerpo o texto sin parser	400	No crea documento
POST con precio 0.5, negativo o "abc"	400	Cantidad sin aumento
POST con precio booleano, arreglo u objeto	400	No se convierte en un precio aceptable
POST JSON mal formado	400	Respuesta JSON, sin traza
Categoría exacta	200	Solo coincidencias; sin coincidencias devuelve []
Categoría repetida como arreglo	400	No se usa como operador MongoDB
GET dos veces	200	Cantidad estable
POST formulario válido	302	Aparece en HTML y API
POST formulario inválido	400	Conserva valores
Base inaccesible durante una operación	500	Mensaje genérico, nunca [] como éxito
Fallo de conexión al iniciar	Sin puerto escuchando	No anuncia disponibilidad
CSS, inicio y ruta inexistente	200, 200 y 404	Continuidad del sitio

Para probar un rechazo de esquema más allá del validador HTTP, usar en una prueba aislada el servicio con nombre vacío o precio no convertible; Mongoose debe rechazarlo. No retirar el middleware del proyecto para hacer la prueba.

Ejecutar los rechazos de la matriz en Postman

Conservar el GET del listado y duplicar el POST para los ensayos de entrada inválida. En Body, raw, JSON, cambiar solo el dato que se prueba, por ejemplo precio a 0.5, negativo, texto no convertible, booleano, arreglo u objeto. Para un cuerpo ausente, seleccionar Body en none. Para JSON mal formado, enviar deliberadamente un cuerpo con sintaxis incompleta manteniendo Content-Type en application/json.

En cada caso revisar 400 y volver al GET sin filtros para comparar cantidades: el rechazo no debe crear documentos. Las pruebas del formulario se realizan en el navegador. Los fallos de arranque y conexión se diagnostican desde el servidor y su configuración; Postman solo puede observar una respuesta si el servidor está atendiendo.

Persistencia sin duplicar la demostración

1. Consultar el listado en Postman, anotar cantidad inicial y crear **una vez** por POST.

2. Guardar ID y Location de la respuesta 201.
3. Dejar terminar la solicitud, cerrar el servidor y volver a iniciarlo con la misma base.
4. En Postman, volver a enviar el GET a la URL guardada desde Location y comparar identidad y campos. No repetir el POST.
5. Listar dos veces y verificar cantidad estable.
6. Verificar en Data Explorer la base `mercado_artesanal_u8` , colección `productos` y `_id` correspondiente.
7. Registrar rechazos con cantidades antes/después. No borrar datos para fabricar un resultado.

El modelo valida y persiste; el servicio traduce operaciones; el controlador espera y decide la respuesta; el router asocia método y dirección. Separar responsabilidades no significa repetir la consulta en todas las capas.

9. Estado final del proyecto

```
mercado-artesanal-http-mongodb/
  src/
    index.js                      (conecta antes de escuchar)
    app.js                        (sitio, API y errores)
    config/
      entorno.js                  (conservado)
      base-datos.js               (conservado, incluye ajuste DNS autorizado)
    modelos/producto.js           (agrega default disponible)
    servicios/
      productos.js                (referencia en memoria, ya no se importa)
      productos-mongo.js           (creación y lectura asíncronas)
    controladores/productos.js     (await y respuestas HTML/JSON)
    rutas/
      productos.js                (HTML, conservado)
      productos-api.js            (API, nuevo)
    middleware/
      productos.js                (validación de entrada)
      solicitudes.js              (conservado)
      errores.js                  (nuevo)
    archivos.js                   (referencia, no participa del arranque)
  scripts/                        (diagnóstico y laboratorio conservados)
  datos/productos.json            (referencia, no se carga automáticamente)
  views/                          (sin cambios)
  public/                         (sin cambios)
  .env.example                    (base de unidad 8, sin secretos)
  .gitignore                      (conservado)
  eslint.config.js                (src y scripts, conservado)
  package.json                    (dependencias y scripts conservados)
  package-lock.json               (conservado o generado por NPM)
```

10. Ticket de salida y transferencia

1. ¿Por qué el puerto se abre después de conectar?
2. ¿Qué problema habría si desconectamos en finally al terminar main?
3. ¿En qué se distingue ID inválido de documento inexistente?
4. ¿Por qué HTML responde 302 y la API 201?
5. ¿Qué cambia al usar lean y cómo preservamos producto.id?
6. ¿Dónde se validan los campos y dónde se mapean los errores a HTTP?

El TP 08 aplica este recorrido a reservas sobre una copia del TP 07. Se entrega individualmente por URL de GitHub el lunes siguiente a las 19:59. No incorpora actualización, borrado, paginación ni relaciones.

Referencias oficiales

- Express 5: manejo de errores
- Mongoose: consultas
- Mongoose: defaults
- Mongoose: validación
- Mongoose: ObjectId estricto)
- Postman: parámetros y cuerpo de las solicitudes
- Postman: estructura de la respuesta